

게임 AWS 클라우드 서비스 활용 예시 현장실습 경험

20192460 홍상준

escatrgot@gmail.com



목차

1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

1. 온프레미스 마이그레이션 (컴퓨팅, 스토리지, 데이터베이스)
2. 글로벌 인프라 (Route 53, CloudFront, Global Accelerator)
3. 스케일업, 스케일아웃 (LoadBalancing, Auto Scaling Group)

2. 게임 장르별 아키텍처 패턴

1. 웹기반 비동기 방식 (L7, HTTP/HTTPS, ALB)
2. 세션 기반 게임 (L4, TCP/IP & UDP, NLB)

3. 아삭 아삭 사과게임

1. API Server (AWS Lambda, ALB, MongoDB Atlas, Firebase)
2. 정적 콘텐츠 글로벌 배포 (S3 + CloudFront)
3. 클라이언트 측 동기화 처리 (Photon Cloud)





게임 회사의 공통 고민

게임 회사의 고민

오버 스펙 서버 → 큰 부담

기존 내수 시장 → 글로벌 런칭

그외 등등.. → 클라우드로 이동



1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

세 가지 핵심 고민 포인트

고민 영역	온프레미스	클라우드
트래픽 변동	미리 많이 사두면 과투자, 적게 준비하면 장애 발생	탄력적 스케일링, 필요할 때만 리소스 확보
글로벌 확장	국가별 서버 구축·운영 부담	글로벌 리전/에지로 저지연 서비스
운영 부담	구축/패치/모니터링에 많은 리소스 필요	관리형 서비스로 게임 개발에 집중



컴퓨팅 마이그레이션

워크로드 분류

동기화 서버 (컴퓨팅 최적화)

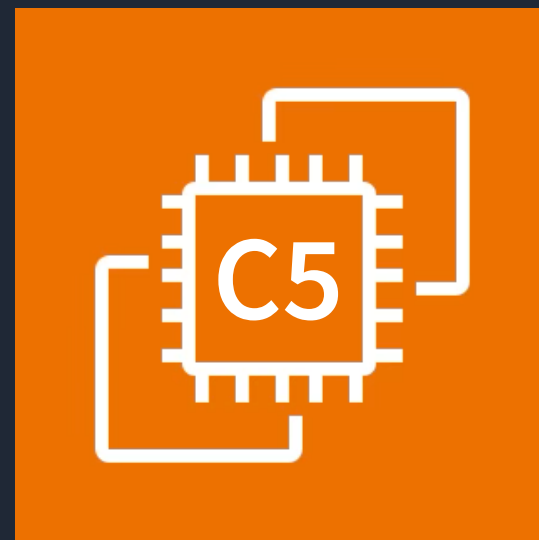
DB 서버 (메모리 최적화)

API 서버 (범용 컴퓨팅, 서버리스)



1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

EC2 & Lambda



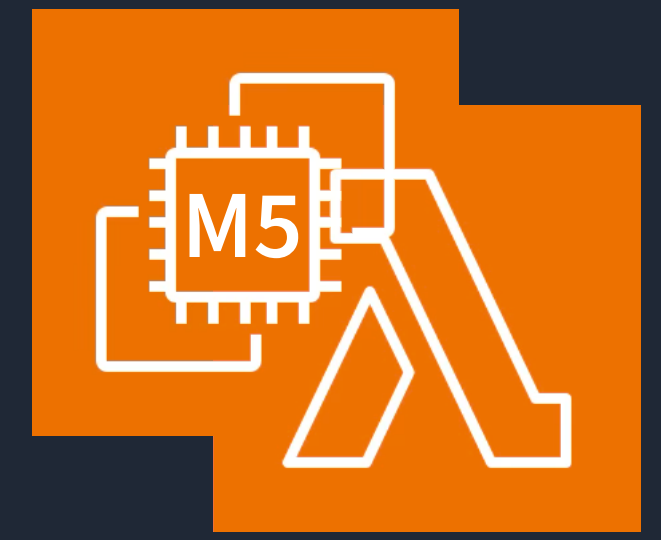
동기화 서버

1. Stateful 네트워크 I/O 처리
 - 패킷 큐
2. CPU 바운드 연산
 - 배치처리 : 패킷 직렬화
 - 게임 로직 : 인풋, 이동, 충돌 판정, 스킬 효과
3. Tick(100ms) 내에 로직 처리 & 상태 브로드캐스트



DB 서버

1. 페이지 캐시, 인덱스, 쿼리 엔진 최적화
2. 페이지 스왑 Cost 감소 → 대용량 데이터 고속 처리
3. 스토리지 성능 → IOPS 성능 향상



API 서버

1. Stateless HTTP 요청-응답 패턴
 2. API 외부에 노출 → L7인터페이스 REST, gRPC, WebSocket,
 3. 로그인, 인벤토리, 상점, 랭킹 등 비즈니스 로직
3. 범용 EC2 VS Lambda
- 상시 작동 X, 포트 리스닝 X
 - Lambda는 서버 X
- API Gateway 협력 필수



스토리지 마이그레이션

스토리지 분류

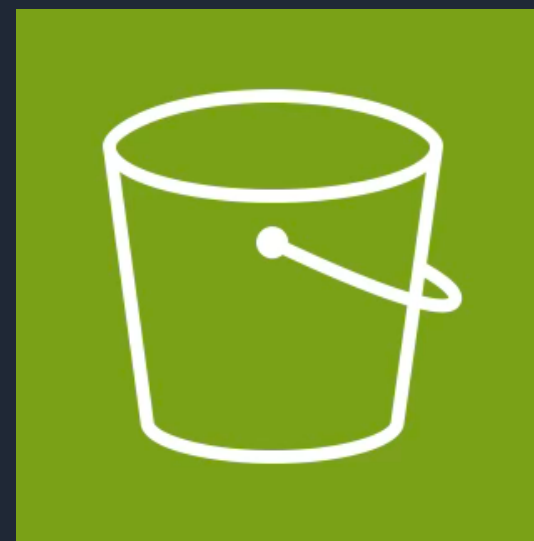
정적 파일 (S3)

동적 데이터 (EBS)



1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

S3 & EBS



파일 서버 & FTP 서버

1. 잘 안 바뀌는 정적 파일을, 무한대에 가깝게
2. HTTP/HTTPS Method
 - DB용은 절대 아님.
3. 게임 예시
 - 클라이언트 빌드 파일, 패치 파일,
 - 에셋, Addressables
 - [텍스처, 사운드, 충돌체(길찾기)]



온프레미스 로컬 SSD/HDD

2. 자주 수정되고 랜덤 I/O가 많은 동적 데이터
2. 자잘한 랜덤 업데이트 + 인덱스 + 쿼리 작업
3. 게임 예시
 - 게임 빌드 실행 파일, 유저, 인벤토리 등..
 - DB 데이터 파일
 - MySQL(.frm, .myd, .myi, .ibd)
 - PostgreSQL

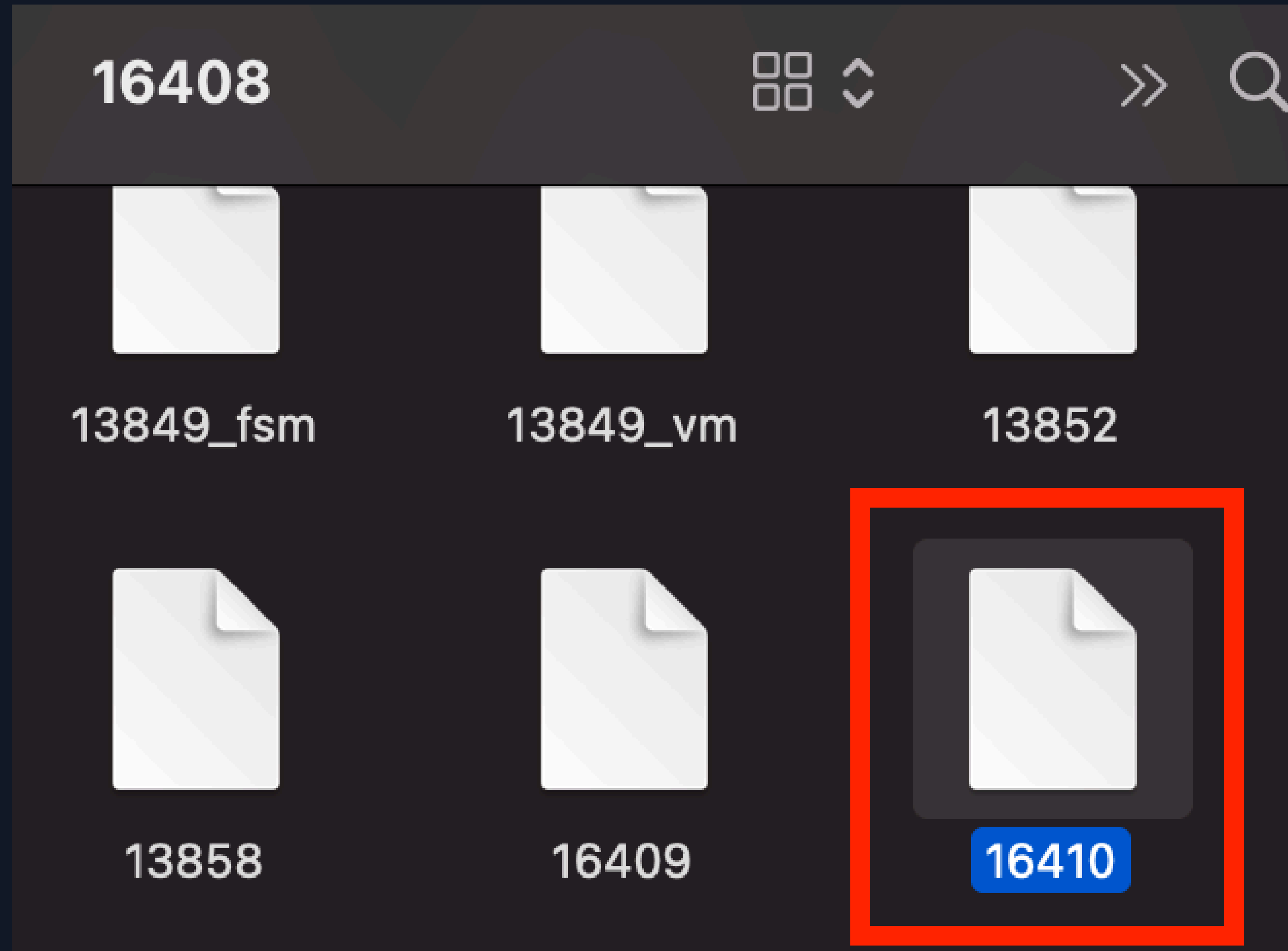


스토리지 마이그레이션

PostgreSQL
SSD에 덤프 파일로 저장



1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스





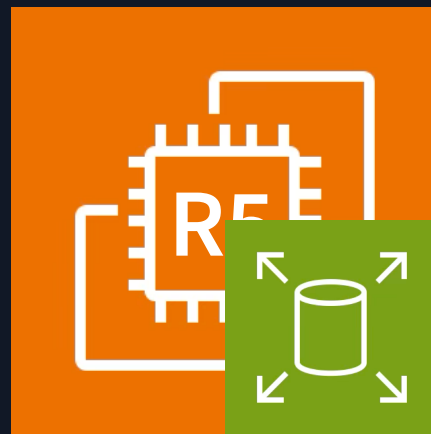
DB 서버 마이그레이션

DB 서버 분류

직접 운영형 : (EC2/EBS/Redis)
관리형 : (RDS/DynamoDB)
영속성 DB : (RDS/DynamoDB)
캐시 DB : (ElasticCache)

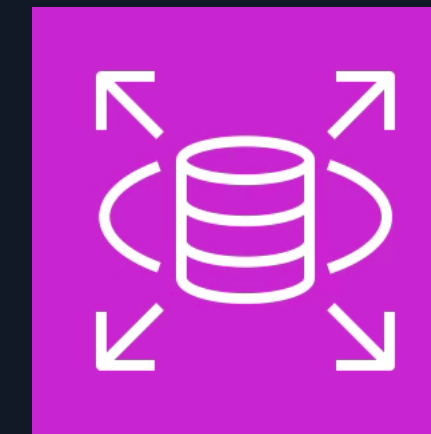


1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

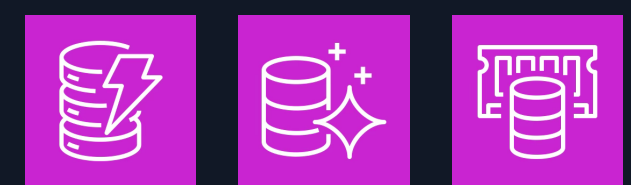


직접 운영형

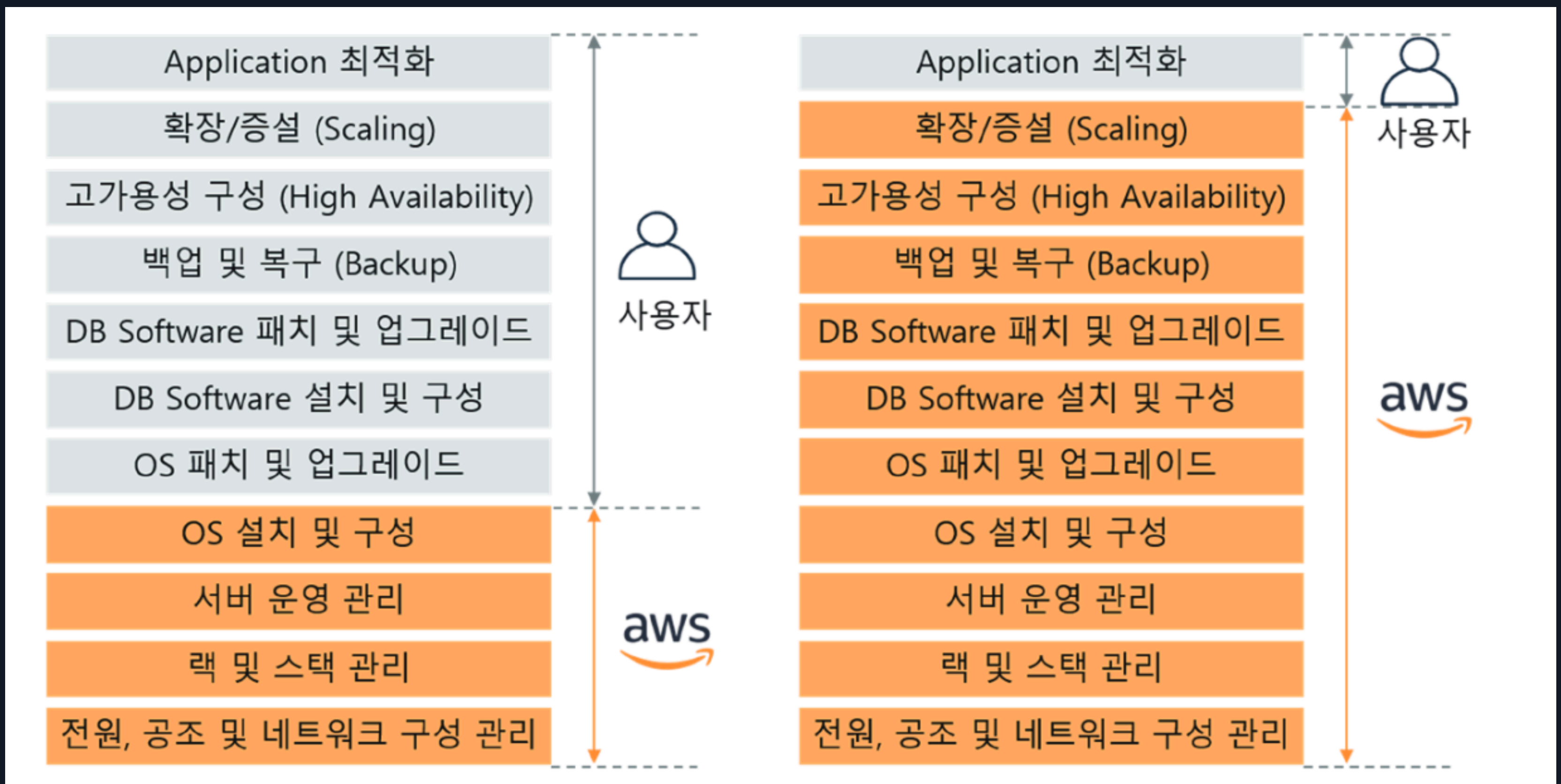
EC2 + EBS + 직접 설치 DBMS/Redis
온프레미스식 구조/튜닝/백업 등 직접 책임



관리형



RDS, Aurora, DynamoDB, ElasticCache
백업, Multi-AZ, 패치 등 인프라 레벨 위임





DB 서버 마이그레이션

관리형 비교

직접 운영형 : (EC2/EBS/Redis)
관리형 : (RDS/DynamoDB)
영속성 DB : (RDS/DynamoDB)
캐시 DB : (ElasticCache)



1. 모듈 개관, 게임 회사의 니즈에 대응되는 AWS 서비스

관리형 DB 서비스 등등..

서비스	설명	특징
Amazon RDS	관리형 관계형 데이터베이스	자동 백업, Multi-AZ 지원
Amazon Aurora	고성능 RDS(MySQL, PostgreSQL 호환)	서버리스 옵션 지원
Amazon DynamoDB	서버리스 NoSQL 데이터베이스	자동 확장, 초고속 성능
Amazon ElastiCache	인메모리 캐시 서비스(Redis, Memcached)	데이터 읽기 속도 향상
Amazon Redshift	데이터 웨어하우스	대규모 데이터 분석 최적화



글로벌 인프라

클라이언트는 엔드포인트 하나만 보지만, 뒤에는 여러 층이 있다



Route 53 / CloudFront / Global Accelerator



Route 53

DNS 서비스로 도메인 ↔ IP 등록

요청을 어느 에지/리전에 라우팅할지 선택 (서울/도쿄/미국 동부 등)



CloudFront

CDN + L7 게이트웨이 역할

S3 정적 파일 캐싱 + REST API 응답 캐싱

WebSocket 프록시 지원으로 양방향 통신 가능



Global Accelerator

L4 기반 TCP/UDP 가속기

에지에서 AWS 백본 타고 가장 건강한 리전의 NLB/ALB로 라우팅

FPS, MOBA 같이 지연에 민감한 게임에 최적화



스케일링 전략과 로드 밸런싱

"게임은 결국 피크 트래픽에 맞춘
스케일 전략 싸움이다."

Auto Scaling Group, Elastic LoadBalancing

↑ Scale Up

- ✓ 더 큰 인스턴스로 교체, 수직 확장
- + 구조가 단순하고 로드밸런서가 필요 없음
- 머신 스펙에는 한계가 있음

↔ Scale Out

- ✓ 서버 수를 늘려 수평 확장
- + 탄력적 확장이 가능하고 이론상 무제한 확장
- 로드 밸런서가 필수적으로 필요함

로드 밸런서 유형



Application Load Balancer

HTTP/HTTPS 트래픽, URL/Host 기반 라우팅,
Lambda 연동
일반 API 서버, 게임 백엔드 웹서비스에 적합



Network Load Balancer

TCP/UDP 프로토콜 지원, 낮은 지연시간
실시간 게임 트래픽 분산에 최적화됨

게임 장르별 아키텍처 패턴



게임 네트워크

네트워크 분류

Stateful VS Stateless
Layers

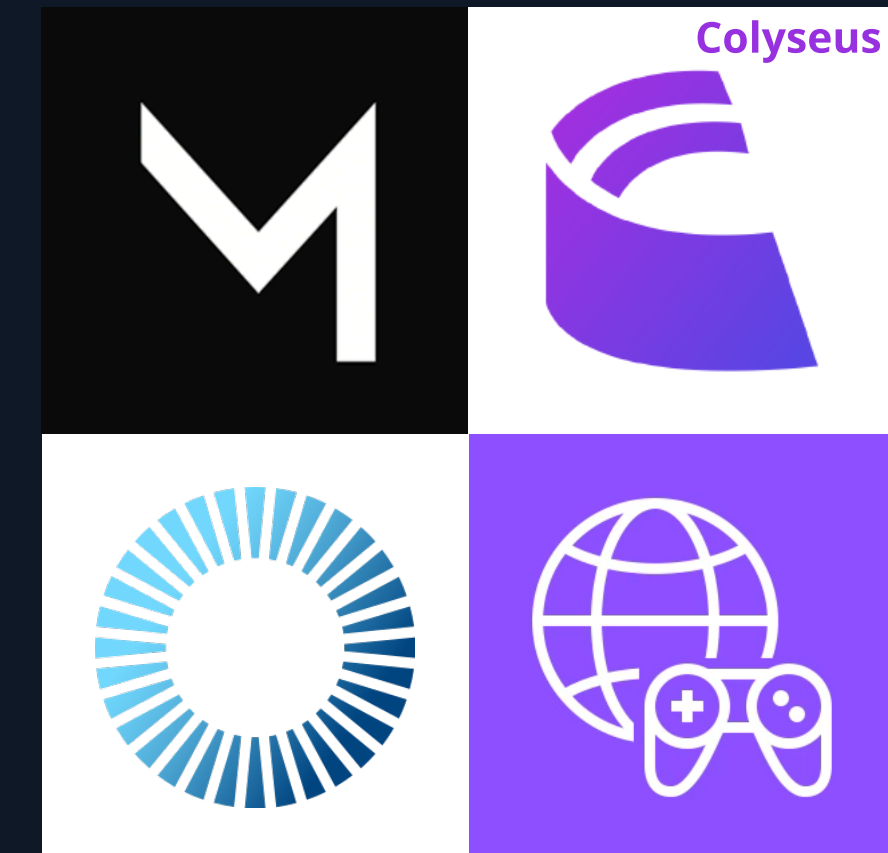
- L4(UDP/TCP)
- L7(HTTP, Websocket)



게임 네트워크는 크게 두갈래로 나뉩니다.

1 세션 기반, stateful, L4(TCP/UDP)

실시간 동기화가 필요한 게임에 사용
서버가 클라이언트 상태를 지속적으로 추적
연결 유지 및 틱 단위 시뮬레이션
예: FPS, MOBA, 액션 RPG 레이드



2 비동기 HTTP, stateless, L7(REST)

단발성 요청-응답 패턴에 최적화
서버는 요청이 올 때만 상태 처리
웹 기술 기반, 스케일링 용이
예: 모바일 RPG, 카드 게임, 메타 시스템



세션 기반 게임 예시들

세션 기반 + 비동기 혼합

PC 플랫폼

- 롤, 오버워치, 배그, 로아

모바일 플랫폼

- 운빨존망겜, 쿠키런 모험의 탑

대표 장르

- FPS, 대전 격투, RTS
- MOBA, 배틀로얄,
- 액션 RPG 레이드 등





세션 기반 게임 다이어그램

세션기반 + 비동기 혼합

Stateless : 로비/상점/인벤토리

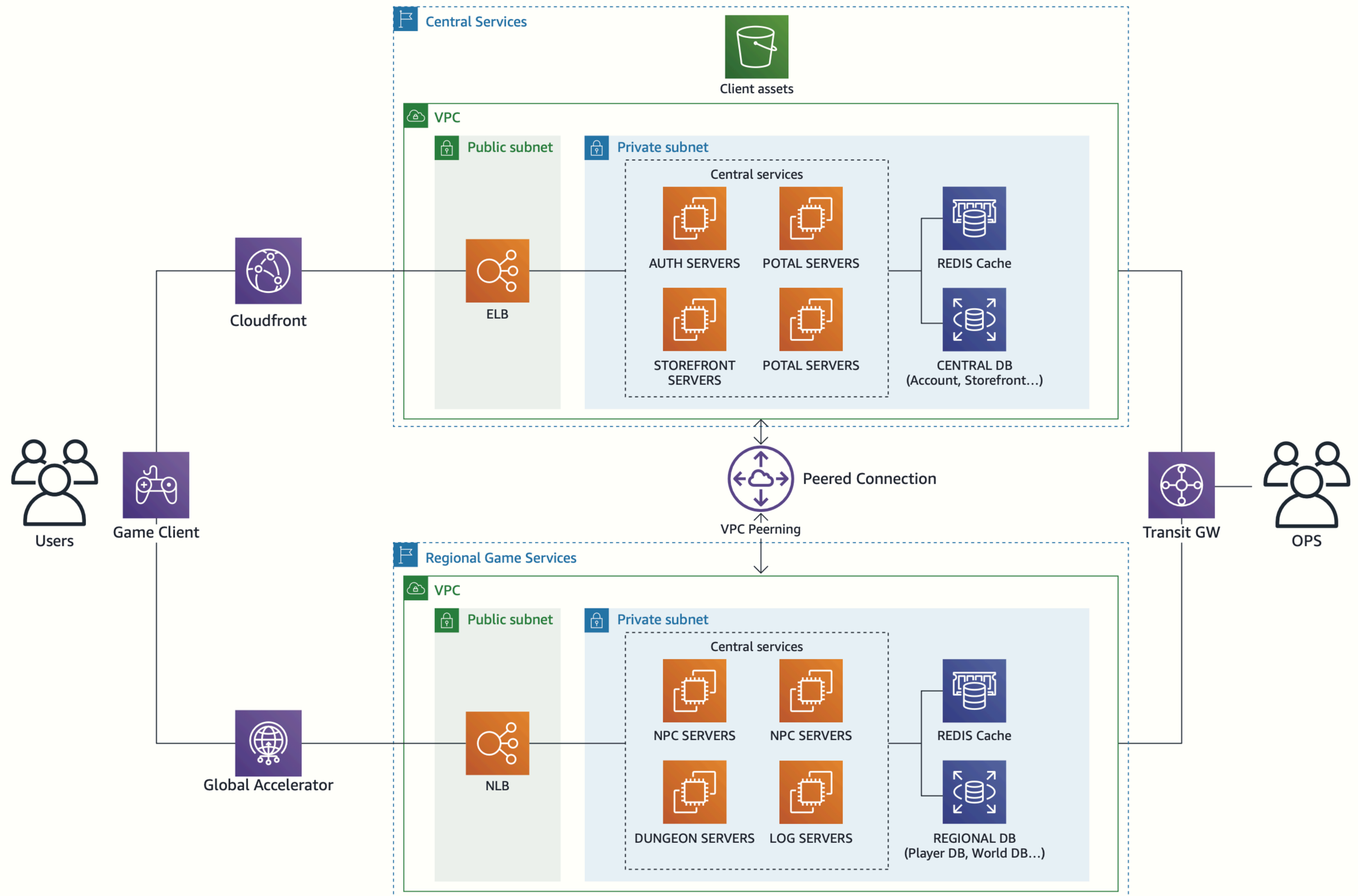
- ELB → 비동기 서비스
- S3 → 콘텐츠

Stateful : 전투/레이드/던전

- NLB → 실시간 서비스



2. 게임 장르별 아키텍처 패턴





비동기 HTTP 게임 예시들

세션기반 + 비동기 혼합

모바일 플랫폼

- 가디언 테일즈, 트릭컬, 카피바라
- 세븐나이츠, 쿠키덤, BTD 6

대표 장르

- 모바일 수집형 RPG, 캐주얼 게임
- 자동 전투, 결과 업로드형 PVP



2. 게임 장르별 아키텍처 패턴





비동기 HTTP 게임 다이어그램

HTTP기반 비동기 방식

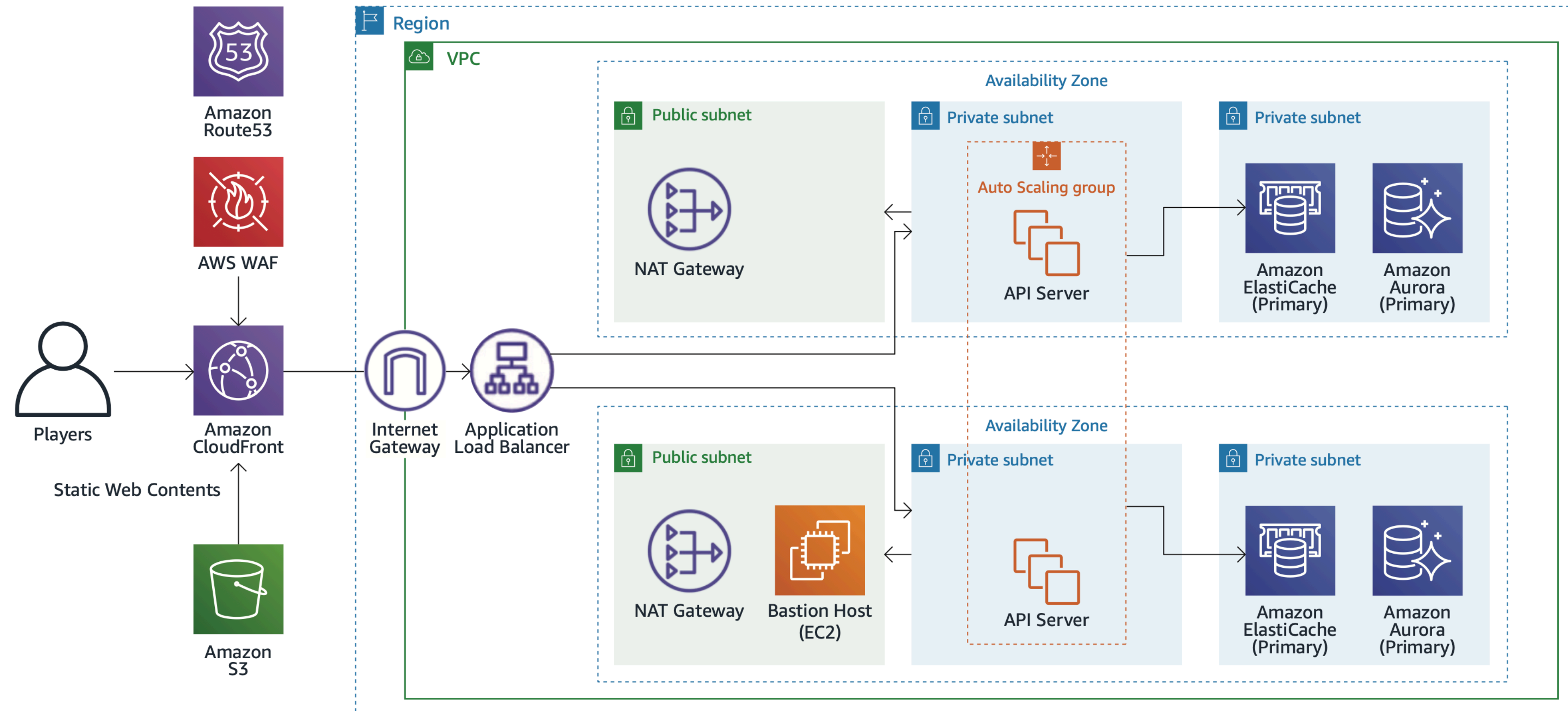
Stateless : 로비/상점/인벤토리

- S3 → 콘텐츠
- Route53 → IP 전달
- CloudFront → 엣지 접근



2. 게임 장르별 아키텍처 패턴

웹 기반 비동기 방식 게임 아키텍처 모범 사례



아삭아삭 사과게임

현장실습 경험



작업 환경 및 요구사항

회사가 사용하던 솔루션

MongoDB Atlas Cloud

Firebase FCM

Airbridge



3. 아삭아삭 사과게임 현장실습

아삭아삭 사과게임 - 친구와 함께 세로모드

Metric Studio Inc.
광고 포함 · 인앱 구매

4.3★
리뷰 68개

1만+
다운로드

전체이용가 ①



회사의 기존 환경

데이터베이스: MongoDB Atlas

푸시 알림: Firebase FCM

유저 분석 : Aridbridge

프로젝트 요구사항

멀티플레이 콘텐츠 : BattleMode, PVP

RestfulAPI

- 유저 관리(아이템, 재화, 친구, 등등..)
- 주간 랭킹
- 친구추가
- 플레이 전적



아삭아삭 사과게임 다이어그램

세션기반 + 비동기 혼합

Stateless

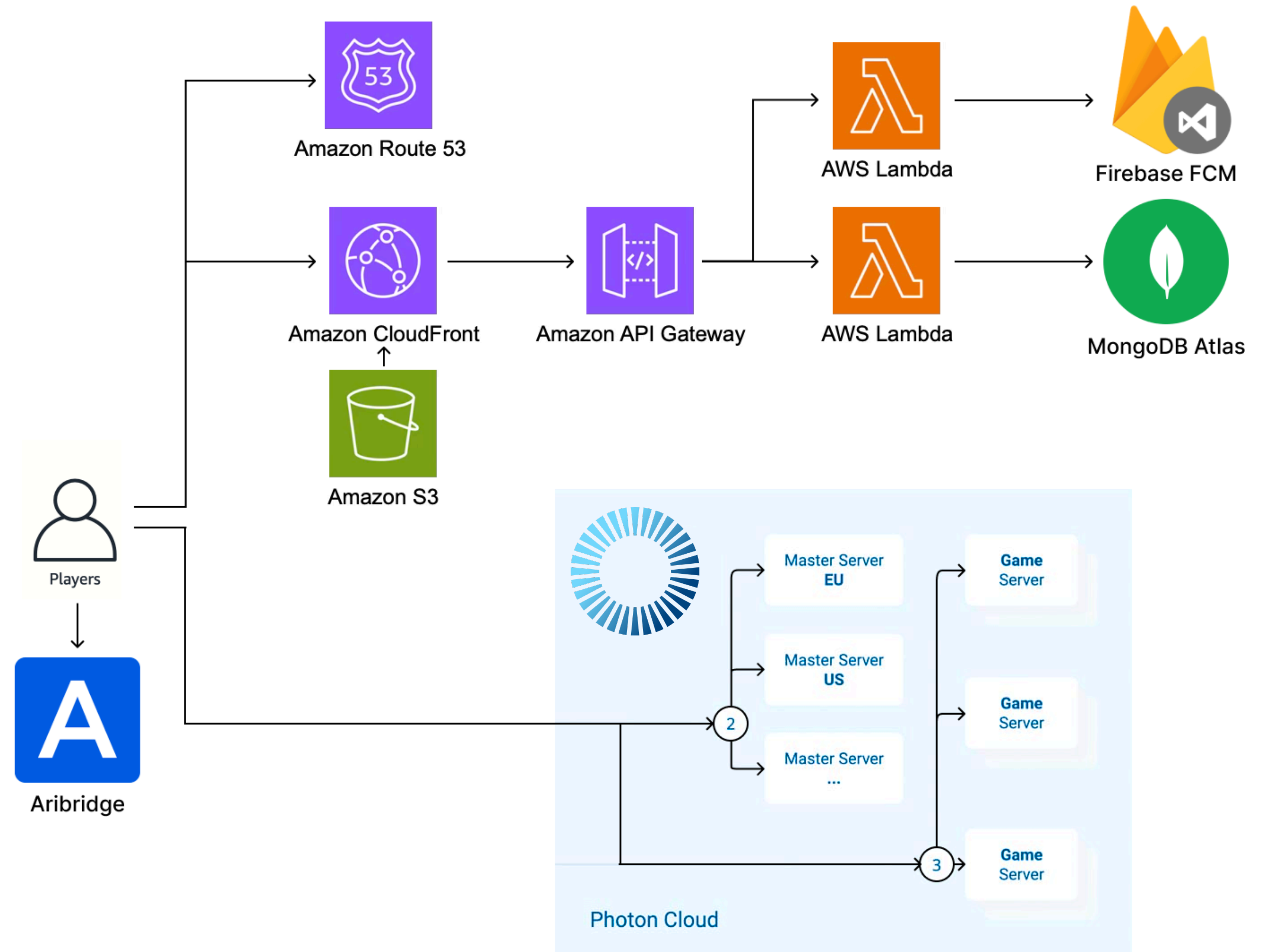
- API Gateway → Lambda
- CloudFront → 정적리소스

Stateful

- Photon → Matchmaking
- Photon → Host-Client



3. 아삭아삭 사과게임 현장실습





API 계층

서버리스 백엔드

AWS Lambda, API Gateway

MongoDB Atlas Cloud

Firebase FCM

Python Chalice



3. 아삭아삭 사과게임 현장실습

선택 이유

풀타임 상주가 필요 없음

관리형 서비스, 소규모팀(본인 포함 2명), 사용한만큼 과금

회사가 이미 MongoDB Atlas를 사용 중인 환경

구조 및 역할

API Gateway → Lambda(Chalice) → MongoDB Atlas 흐름

유저 관리: 회원가입, 로그인, 프로필 정보 관리

친구 관리: 친구 조회, 추가, 푸시알림

랭킹/점수: 주 단위 랭킹 접근, 랭킹 종료 알림

우편함 : 우편 데이터 추가, 삭제



MongoDB Firebase 예시

서버리스 백엔드

AWS Lambda, API Gateway

MongoDB Atlas Cloud

Firebase FCM

Python Chalice



3. 아삭아삭 사과게임 현장실습

```
def put_pending_friend(self, input_data: PutRedirectedGuestRequest) :  
    ...  
    invite_owner_user = UserModel.get(input_data.invite_owner_id)  
    if UserValidator.check_idempotent(input_data, invite_owner_user):  
        invite_owner_user.friends.append(input_data.redirected_guest_id)  
        invite_owner_user.update_friend()  
        message_body = PushMessage.add_pending_friend(input_data)  
        PushMessage.send_fcm_notification(  
            invite_owner_user.device_id, message_body)  
    ...  
  
def send_fcm_notification(device_token: str, body_text: str):  
    ...  
    notification = messaging.Notification(  
        title='친구 추가', body=body_text,)  
    message = messaging.Message(  
        notification=notification, token=device_token,  
        data={'type': 'FRIEND_ADDED'},)  
    response = messaging.send(message)  
    ...
```



리소스 계층

정적 리소스 배포

Unity Addressable

AWS S3

CloudFront, Route53

AWS CLI



3. 아삭아삭 사과게임 현장실습

선택 이유

스토리지 : 정적 리소스 (Addressable) 저장

전송 최적화 : CDN 구성, 낮은 전송 지연, 높은 전송 속도

무패치 리소스 수정 : 리소스를 번들 단위로 관리

구조 및 역할

Addressable Build → CLI (S3 cp) → CLI (CloudFront)

에셋 번들 관리 : 메모리에 에셋 적재 및 해제 취소 선택으로 Memory 최적화

텍스처, 애니메이션, 파티클 이펙트, 다국어 텍스트 = 약 300MB



S3 CloudFront CLI 자동화 예시

정적 리소스 배포

Unity Addressable

AWS S3

CloudFront, Route53

AWS CLI



3. 아삭아삭 사과게임 현장실습

```
if [ $# -lt 1 ]; then
    echo "\$1 : 새로운 어드레서블 버전을 기입해 주세요!"
    echo "예시 : sh update-addressable.sh <version_name>"
    exit 1
fi

version_name="$1"

# Backup to Submodule
sh backup-addressable.sh ${version_name}

# Copy To AWS S3
aws s3 cp --recursive ServerData/${version_name}/Android/ ${APPLE_GAME_S3_ANDROID_PATH}
aws s3 cp --recursive ServerData/${version_name}/iOS/ ${APPLE_GAME_S3_IOS_PATH}
aws cloudfront create-invalidation \
    --distribution-id ${PRIVATE_ID} \
    --paths "/${GAME_PROJECT_NAME}*"

```



실시간 세션

멀티 플레이 구현
Photon PUN Cloud



3. 아삭아삭 사과게임 현장실습

구조 및 역할

1:1 매치 기반 실시간 세션 서버

매치메이킹 + 상태 동기화

RPC 기반 이벤트 전파

Host-Client 방식의 연결 구조

Photon MatchMaking API

```
PhotonNetwork.IsConnectedAndReady  
PhotonNetwork.CreateRoom(roomCode, roomOptions);  
PhotonNetwork.JoinRoom(roomCode);  
PhotonNetwork.JoinRandomRoom()  
PhotonNetwork.LeaveRoom();  
PhotonNetwork.LeaveLobby();  
PhotonNetwork.Disconnect();
```



Photon PUN RPC 인풋 동기화

멀티 플레이 구현
Photon PUN Cloud



3. 아삭아삭 사과게임 현장실습

```
[PunRPC]
void SyncDragStart(Vector2 receivedStartPos)
{
    _otherStartPos = receivedStartPos;
}

[PunRPC]
void SyncDragMove(Vector2 receivedStartPos, Vector2 receivedEndPos)
{
    _otherStartPos = receivedStartPos;
    _otherEndPos = receivedEndPos;
    DrawVisual(_otherBoxVisual, _otherStartPos, _otherEndPos); // 상대방 박스 업데이트
    DrawSelection(_otherStartPos, _otherEndPos, ref _otherSelectionBox);
    SelectApples(_otherSelectionBox, _otherSelectedApples, false);
}

[PunRPC]
void SyncDragEnd()
{
    CalculateOthers(_otherSelectedApples);
    _otherStartPos = Vector2.zero;
    _otherEndPos = Vector2.zero;
    DrawVisual(_otherBoxVisual, _otherStartPos, _otherEndPos); // 상대방 박스 리셋
}
```